

Chapter 4 作业

学号：23336128

姓名：梁力航

大学数据库模式 (Figure 1)

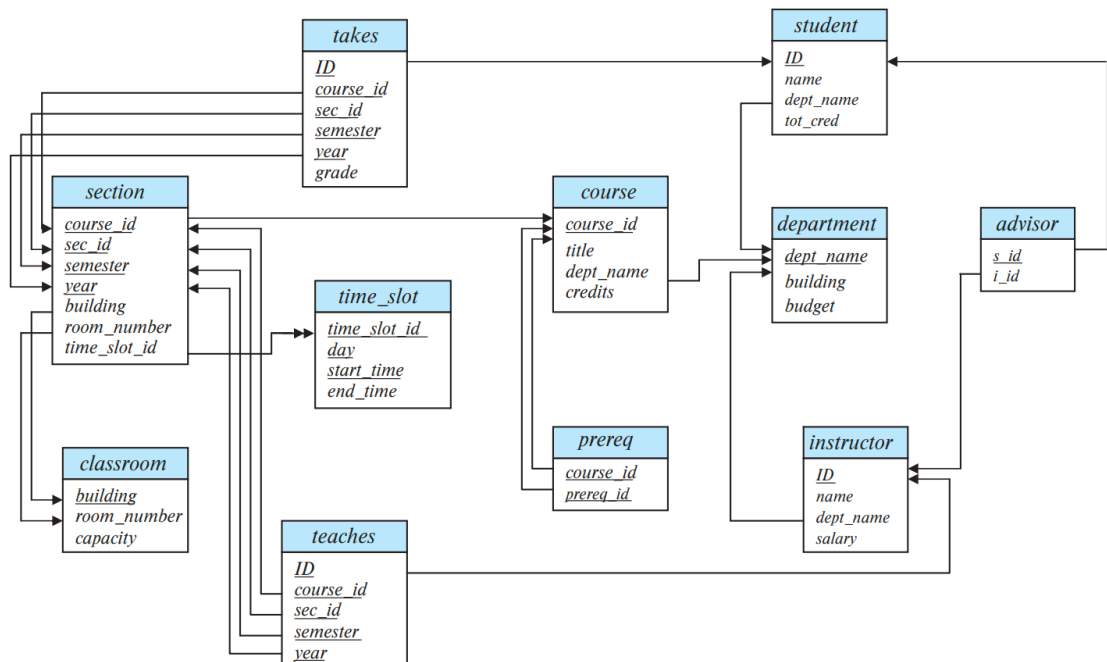


Figure 1 Schema diagram for the university database.

Figure 1 Schema diagram for the university database.

问题1：外连接的等价重写

外连接表达式可以在SQL中不使用SQL outer join操作来计算。为了说明这一点，展示如何重写以下SQL查询而不使用外连接表达式：

a) 重写左外连接

左外连接的核心思想是保留左表的所有记录，即使右表中没有匹配的记录。我们可以通过联合（UNION）两部分来实现：第一部分是内连接的结果，第二部分是左表中没有匹配的记录（用NULL填充右表的列）。

原查询：

```
select * from student natural left outer join takes
```

重写后：

```
-- 第一部分：内连接，获取有匹配的记录
select student.ID, student.name, student.dept_name, student.tot_cred,
       takes.course_id, takes.sec_id, takes.semester, takes.year, takes.grade
from student
join takes on student.ID = takes.ID

UNION

-- 第二部分：左表中没有匹配的记录，右表列用NULL填充
select student.ID, student.name, student.dept_name, student.tot_cred,
       NULL as course_id, NULL as sec_id, NULL as semester, NULL as year, NULL as grade
from student
where student.ID not in (select ID from takes)
```

b) 重写全外连接

全外连接需要保留两个表的所有记录。这可以看作是左外连接和右外连接的组合。我们需要三部分：内连接的结果、左表独有的记录、右表独有的记录。

原查询：

```
select * from student natural full outer join takes
```

重写后：

```
-- 第一部分：内连接，获取两表都有匹配的记录
select student.ID, student.name, student.dept_name, student.tot_cred,
       takes.course_id, takes.sec_id, takes.semester, takes.year, takes.grade
from student
join takes on student.ID = takes.ID

UNION

-- 第二部分：student表中没有匹配的记录
select student.ID, student.name, student.dept_name, student.tot_cred,
```

```

        NULL as course_id, NULL as sec_id, NULL as semester, NULL as year, NULL as grade
from student
where student.ID not in (select ID from takes)

UNION

-- 第三部分: takes表中没有匹配的记录
select takes.ID, NULL as name, NULL as dept_name, NULL as tot_cred,
        takes.course_id, takes.sec_id, takes.semester, takes.year, takes.grade
from takes
where takes.ID not in (select ID from student)

```

问题2：级联删除的影响分析

SQL允许外键依赖引用相同的表，如以下示例中所示：

```

create table manager
(employee_ID char(20),
manager_ID char(20),
primary key employee_ID,
foreign key (manager_ID) references manager(employee_ID)
on delete cascade );

```

在这里，"employee_ID" 是"manager"表的主键，这意味着每个员工最多只有一个经理。外键子句要求每位经理也必须是一名员工。请解释当"manager"关系中的元组被删除时会发生什么事情。

回答：

当删除manager表中的一条记录时，由于设置了"on delete cascade"，会触发级联删除操作。具体来说，如果删除的是某个员工的记录，那么所有以这个员工作为经理的其他员工记录也会被删除。这个过程会递归进行，形成一个删除链。

举个例子，假设员工A是员工B的经理，员工B又是员工C的经理。如果我们删除员工A的记录，那么首先会触发级联删除，删除所有manager_ID为A的记录（即员工B）。而删除员工B又会触发新的级联删除，删除所有manager_ID为B的记录（即员工C）。这样一来，删除一个高层管理者会导致整个管理层级下的所有员工记录都被删除。

问题3：创建视图

定义一个名为"tot_credits"的视图（year, num_credits），该视图显示每年所修的总学分数。

SQL语句:

```
create view tot_credits(year, num_credits) as
  select section.year, sum(course.credits) as num_credits
  from section
  join takes on section.course_id = takes.course_id
             and section.sec_id = takes.sec_id
             and section.semester = takes.semester
             and section.year = takes.year
  join course on section.course_id = course.course_id
  group by section.year;
```

这个视图通过连接section、takes和course三个表来计算每年的总学分。首先，我们需要通过takes表找到学生实际选修的课程，然后从course表中获取每门课程的学分，最后按年份分组并求和。这样就能得到每年所有学生选修课程的总学分数。

问题4：不使用子查询和集合操作的查询重写

请表达以下查询的SQL，不使用子查询和集合操作（参考fig 1）。

原查询:

```
select ID
from student
except
select s_id
from advisor
where i_ID is not null;
```

重写后:

```
select distinct student.ID
from student
left join advisor on student.ID = advisor.s_id and advisor.i_ID is not null
where advisor.s_id is null;
```

原查询的目的是找出所有没有导师的学生。我们可以用左外连接来实现：将student表与advisor表进行左外连接，连接条件是学生ID匹配且导师ID不为空。对于那些没有导师的学生，左外连接会在advisor的列中填充NULL。因此，我们只需要筛选出advisor.s_id为NULL的记录，就能得到所有没有导师的学生。

问题5：使用USING条件代替NATURAL JOIN

重新编写查询，使用内连接和使用"using"条件来代替"natural join"（参考fig1）：

原查询：

```
select *  
from section natural join classroom;
```

重写后：

```
select *  
from section  
join classroom using (building, room_number);
```

通过观察Figure 1中的数据库模式，我们可以看到section表和classroom表有两个共同的属性：building和room_number。natural join会自动在这两个属性上进行连接。使用USING子句可以明确指定连接的列，这样做的好处是更加清晰明了，避免了natural join可能带来的歧义。而且，USING子句在结果中只会保留一份连接列，这与natural join的行为是一致的。

问题6：授权图中的循环分析

假设用户A对关系r具有所有授权权限，并将关系r的select权限授予public，并附带授权选项。假设用户B随后将r的select权限授予A。这样会在授权图中产生一个循环吗？请解释原因。

回答：

这个场景不会在授权图中产生真正意义上的循环。用户A作为关系r的所有者，本身就拥有所有权限，包括select权限。当A将select权限授予public（附带授权选项）时，所有用户（包括用户B）都获得了这个权限，并且可以将这个权限传播给其他用户。

接下来，用户B将select权限授予A。从表面上看，这似乎形成了一个循环：A授权给public（包括B），B又授权给A。但实际上，A作为关系r的所有者，其权限是固有的，不依赖于任何其他用户的授权。B授予A的权限是冗余的，因为A本来就拥有这个权限。

在授权图中，我们会看到从A到public的一条边，以及从B（作为public的成员）到A的一条边。但这不构成真正的循环依赖，因为A的权限来源是其作为所有者的身份，而不是B的授权。如果我们撤销B对A的授权，A仍然保有其作为所有者的所有权限。